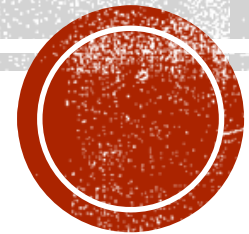


FUNCTION OVERLOADING





PRACTICE EXERCISE

Absoulte.cpp

EXERCISE

Write a program to find the absolute value of an integer.

EXERCISE

```
int absolute(int x)
{
    if (x < 0)
        return -x;
    return x;
}

int main()
{
    cout << absolute(-10);
}
```

EXERCISE

Write a program to find the absolute value of a decimal number.

EXERCISE

```
double absoluteDecimal(double decimal)
{
    if (decimal < 0)
        return -decimal;
    return decimal;
}

int main()
{
    cout << absolute(-10.45);
}
```

**Do we really need different names when
both functions do the same job?**

```

1  #include <iostream>
2  using namespace std;
3
4  int abs(int i);
5  double abs(double d);
6  long long abs(long long l);
7
8  int abs(int i){
9      cout<<"Integer"<<endl;
10     return i < 0 ? -i : i;
11 }
12
13 double abs(double d){
14     cout<<"Double"<<endl;
15     return d < 0.0 ? -d : d;
16 }
17
18 long long abs(long long l) {
19     cout<<"Long"<<endl;
20     return l < 0 ? -l : l;
21 }

```

```

23 int main(void)
24 {
25     cout << abs(-11) << endl;
26     cout << abs(-22.2) << endl;
27     cout << abs(-214748364756) << endl;
28 }

```

Integer

11

Double

22.2

Long

214748364756

 Process exited after 0.04974 seconds with return value 0
 Press any key to continue . . .

WHAT IS FUNCTION OVERLOADING?

- The C language forces us to use different names for similar functions.

- Example:

```
int abs(int);  
double fabs(double);  
long labs(long);
```

- But C++ lets us use just one name for all similar functions.
 - Function overloading is a feature in C++
- Allows two or more functions to have the same name with different parameters.
- This is an example of polymorphism feature in C++.

FUNCTION OVERLOADING

- Defining two or more functions using the same name but different parameters.
- Overloaded functions must differ in their parameter lists
 - must have either different numbers of parameters or
 - different types of parameters.
- When an overloaded function is called, the **compiler uses the argument list** to determine which function of that name is being invoked

```
int abs(int i);  
double abs(double d);  
long abs(long l);
```

FUNCTION OVERLOADING

```
int getValue()  
{  
    return 1;  
}  
  
float getValue()  
{  
    return 1.2;  
}
```



Compilation Error

FUNCTION OVERLOADING & RETURN TYPES

- Functions can not be overloaded if they differ only in the return type.

```
int getValue(int i);  
float getValue(int i);
```

```
// This attempt to overload will fail
```

FUNCTION OVERLOADING

```
float getSquare(float f);  
double getSquare(double d);
```

```
//Function call  
cout << getSquare(10);
```



Compilation Error

AVOIDING AMBIGUITY

- When overloading functions, it is possible to create **ambiguous situations** where the compiler is unable to choose between two or more overloaded functions.
- If there are Ambiguous statements the compilation errors will occur.

Implicit type conversion

```
float getSquare(float f);  
double getSquare(double d);
```

```
//Function call  
cout << getSquare(10);
```

DEFAULT ARGUMENTS

- Default arguments are one possible cause of ambiguity.

```
void addPerson(char* firstName, char* lastName, int age = 0);  
  
void addPerson(char* firstName, char* lastName);
```

- Attempt to call the second version of **addPerson()** will be ambiguous.

```
addPerson("Jane", "Citizen");
```

RESOLVING CALLS TO OVERLOADED FUNCTIONS

The compiler uses the following overload resolution rules to select the best function:

- Exact match (preferred)
 - If a function matches the arguments exactly, it is selected.
- Standard promotions
 - e.g., `char` $\rightarrow$ `int`, `float` $\rightarrow$ `double`
- Standard conversions
 - e.g., `int` $\rightarrow$ `float`, `float` $\rightarrow$ `int`, `int` $\rightarrow$ `double`
- User-defined conversions
 - e.g., conversion constructors or overloaded type conversion operators

HOME WORK

Write a program that calculates the area of different geometric shapes (circle, rectangle, and triangle) using function overloading.

Select a shape to calculate the area:

1. Circle
2. Rectangle
3. Triangle

Enter your choice: 1

Enter the radius of the circle: 5

Area of the circle: 78.54

Select a shape to calculate the area:

1. Circle
2. Rectangle
3. Triangle

Enter your choice: 2

Enter the length and breadth of the rectangle: 4 6

Area of the rectangle: 24.00

Select a shape to calculate the area:

1. Circle
2. Rectangle
3. Triangle

Enter your choice: 3

Enter the base and height of the triangle: 5 7

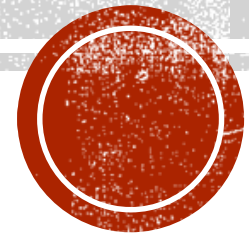
Area of the triangle: 17.50

FUNCTION OVERLOADING IN SUMMARY

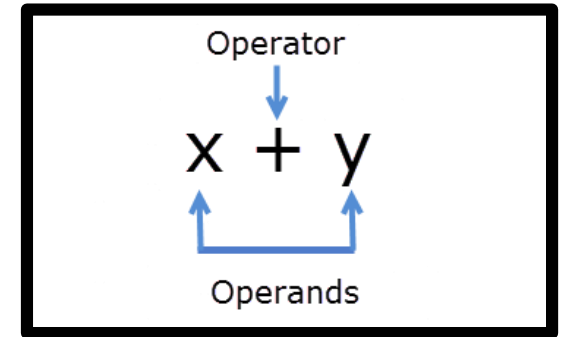
- Define multiple definitions for the same function name in the same scope.
- The definition of the function must differ from each other by the types and/or the number of arguments in the argument list.
- Cannot overload function declarations that differ only by return type.

OPERATOR OVERLOADING

What is meant by Operator Overloading?



WHY—OPERATOR OVERLOADING



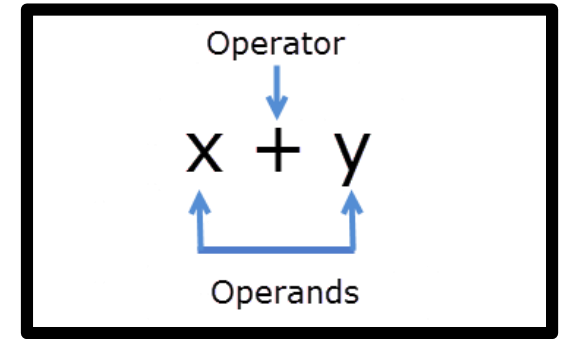
- Operators work with operands.
- Increment operator can be used with int values or int type variables.
Example: count ++.

21

Process exited after 0.07273 seconds with return value 0
Press any key to continue . . . |

```
1  #include<iostream>
2  using namespace std;
3
4  int main(){
5
6      int i, j;
7
8      j = 10;
9      i = (j++, j+100, j+10);
10
11     cout << i;
12
13     return 0;
14 }
```

WHY—OPERATOR OVERLOADING



- Operators work with operands.
 - E.g: The increment operator (++) can be used with arithmetic type variables. It increases the value of the variable by 1 (count ++).

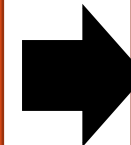
Question: Can we use the above operator with objects directly?

Example: Create an object (N1) from Numbers class and increment all the variables inside the class?

We can't use operators on objects as they are NOT applied for classes.

However, these operators can be overloaded to get special functions done.

```
class Numbers{  
    int x, y;  
  
    public:  
    Numbers():x(0),y(0){  
    }  
};
```



```
int main(){  
    Numbers N1;  
    ++N1;  
}
```



EXERCISE

My Name

Process exited after 0.2586 seconds with return value 0
Press any key to continue . . . |

```
1  #include<iostream>
2  using namespace std;
3
4  int main()
5  {
6      string x="My ";
7      string y="Name";
8
9      cout<<endl<<x+y;
10 }
```

WHAT IS OPERATOR OVERLOADING

- In C++, we can make operators to work for user defined classes.
- Allows to change the use of operators such as +, -, ++, & etc.
- Applied for user defined classes to create the own code to give them new functionality.

Example 1: Use '+' operator to concatenate two strings together

`str1 = str2 + str3`, without calling `append()` function.

Example 2: Overload '++' operator not to increment value by 1, but, by 100.

HOW TO DEFINE OPERATOR OVERLOADING

- Operators are overloaded by defining an operator function.
 - An operator function defines the specific operations that the overloaded operator will perform relative to the class it is designed to work on.
- We must use keyword “operator” before the operator we want to overload
- Works as a function therefore the return type must be defined.

Syntax: operator function

```
returnType operator operatorSymbol(parameter)
{
    // ...
}
```

- BEST PRACTISE: Although you are free to perform any operation you want inside an operator function it is usually best to stay within the context of the operator's normal use.

DIFFERENCE BETWEEN OPERATOR FUNCTIONS AND NORMAL FUNCTIONS

SIMILARITY:

- Operator functions are same as normal functions.

DIFFERENCE:

- Name of an operator function is the operator keyword followed by symbol of operator
- Operator functions are called when the corresponding operator is used.

Operator Function: `Nl++;`

Normal Function: `Nl.numIncrement();`

CAN WE OVERLOAD ALL OPERATORS?

NO.

Operators that can NOT be Overloaded:

- The dot operator (.),
- The scope resolution operator (::),
- The ternary conditional operator (? :) and
- The **sizeof** operator , etc.

WHY? (H/W)

RESTRICTIONS ON OPERATOR FUNCTIONS

- New operators cannot be created.
- Only existing built-in operators can be overloaded.
- An operator's precedence cannot be changed.
- An operator's associativity cannot be altered.
- Cannot change the number of operands that an operator takes.
- Operator functions cannot have default arguments.
- Some operators are by default overloaded.
 - Assignment (=)
 - Address (&)
 - Comma (,)

TYPES OF OPERATOR OVERLOADING

There are **three approaches**

- Overloading unary operator.
- Overloading binary operator.
- Overloading operators using a friend function.

What is a unary operator?

- Operators act on only one operand.
- Examples: increment (++), decrement (--), minus (-) : -33.

What is a binary operator?

- Operators act on two operands.
- Examples: +, -, <=, >, ==,

OVERLOADING UNARY OPERATOR

- No arguments should be passed.
- Works only with one class objects.
- Overloading an operator operating on a single operand.

OVERLOADING UNARY OPERATOR: EXAMPLE 1

```
class Number{
    int num;

    public:
    Number(int a):num(a){
    }

    void display(){
        cout<<"Number variable: "<<num<<endl;
    }

    void operator -(){
        this->num=-num;
    }

};
```

```
int main()
{
    Number myNum1(6);
    -myNum1;

    myNum1.display();
}
```

OVERLOADING UNARY OPERATOR: EXAMPLE 2

```
class Counter{
    unsigned int count;

public:
    Counter():count(0) //constructor
    { }

    void operator++(){
        ++count;
    }

    int getCounter()
    {
        return count;
    }
};
```

Prefix Increment

```
int main()
{
    Counter c1;
    ++c1;           //Prefix

    cout<<"Count= "<<c1.getCounter()<<endl;
}
```

OVERLOADING UNARY OPERATOR: EXAMPLE 3

Postfix Increment

The variable is incremented after its value is used in the expression

Not an Argument



```
Counter operator++(int) {    //postfix increment
    return Counter(count++);
}
```

THE INCREMENT OPERATOR OVERLOADING

The general forms of the prefix and postfix **operator++()** functions

```
// Prefix increment operator ++ObjectName  
returnType ClassName::operator++()  
{  
    // ...  
}
```

```
// Postfix increment operator ObjectName++  
returnType ClassName::operator++(int)  
{  
    // ...  
}
```

THE DECREMENT OPERATOR OVERLOADING

The general forms of the prefix and postfix **operator--()** functions

```
// Prefix decrement operator  --ObjectName  
returnType ClassName::operator--()  
{  
    // ...  
}
```

```
// Postfix decrement operator  ObjectName--  
returnType ClassName::operator--(int)  
{  
    // ...  
}
```

EXERCISE

```
1  #include<iostream>
2  using namespace std;
3
4  class Number{
5      int x,y;
6
7      public:
8          Number():x(0),y(0){}
9
10         void operator++(){
11             x=x+100;
12             y=y+100;
13         }
14
15         void display(){
16             cout<<"X is "<<x;
17             cout<<"\nY is "<<y<<endl;
18         }
19     };
20
```

```
21  int main(){
22
23      Number N1,N2;
24      N1.display();
25
26      ++N1;
27      N1.display();
28      N2.display();
29  }
30
```

```
X is 0
Y is 0
X is 100
Y is 100
X is 0
Y is 0

-----
Process exited after 1.042 seconds with return value 0
Press any key to continue . . . |
```